

Capítulo 5

Abstração



Fonte: Gamma – IA | Adaptada

5.1 Definição e Importância



A abstração é um dos alicerces da Orientação a Objetos. Ela nos permite lidar com a complexidade dos sistemas, concentrando o foco nos aspectos mais relevantes de um problema, sem se perder em detalhes desnecessários. Ao abstrair, você filtra o “ruído” e mantém apenas as informações essenciais para modelar as entidades do mundo real em software. Isso torna a codificação mais clara, facilita a manutenção e ajuda a criar sistemas escaláveis. Neste capítulo, mergulharemos no conceito de abstração, exploraremos como implementá-lo em Java utilizando classes abstratas e interfaces, e veremos como representá-lo visualmente em diagramas UML.

5.1.1 O que é Abstração?

Abstração, no contexto da Orientação a Objetos, é o processo de identificar as características fundamentais de um objeto, ignorando detalhes supérfluos. Se considerarmos um carro, existem inúmeros elementos que poderíamos descrever: o material do volante, a quantidade de parafusos no motor, o tipo de tecido do banco. No entanto, quando pensamos em abstrair o conceito “Carro” em um software, costumamos nos concentrar nas características essenciais como “marca”, “modelo” e “métodos para acelerar ou frear”.

5.1.2 Por que a Abstração é Importante?

- **Redução da Complexidade:** Ao abstrair, você esconde detalhes internos, o que torna o entendimento do sistema mais simples.
- **Facilidade de Manutenção:** Menos detalhes irrelevantes significa menos esforço para ajustar o código no futuro.
- **Reutilização de Código:** Com uma boa abstração, é mais fácil criar estruturas gerais que podem ser especializadas para diferentes contextos.
- **Clareza no Design:** Os programadores compreendem rapidamente as partes importantes de uma classe ou interface, acelerando o desenvolvimento colaborativo.

5.1.3 Exemplo no Mundo Real

Pense em um console de videogame. O jogador aperta botões sem se preocupar com os circuitos internos, transmissões de sinal ou algoritmos de detecção de movimento do controle. Esse “esconderijo” dos detalhes técnicos é a abstração em ação. No código, você pode ter uma classe Controle com métodos como `pressionarBotaoA()` ou `moverDirecional()`, sem precisar explicar como cada comando é interpretado internamente.

5.2 Implementação da Abstração em Java



A abstração é um dos pilares da Orientação a Objetos e pode ser entendida como o processo de esconder os detalhes complexos de uma implementação e fornecer uma interface simples para o usuário. Em Java, a abstração é geralmente alcançada através de classes abstratas e interfaces. Ao criar uma classe abstrata, você define métodos que podem ser implementados pelas subclasses, mas sem especificar como esses métodos devem ser executados, permitindo que cada classe filha forneça sua própria implementação. Isso facilita a organização do código, permitindo que o desenvolvedor se concentre apenas no que a classe faz, e não em como cada parte do processo é realizada. Por exemplo, imagine uma classe abstrata chamada "Animal", com um método abstrato "emitirSom()". Cada animal concreto, como "Cachorro" ou "Gato", implementa o método "emitirSom()" de maneira diferente, mas a interface para o usuário permanece a mesma, simplificando o código.

No contexto de um sistema mais amplo, a abstração ajuda a manter o código mais modular e reutilizável. Ao se concentrar apenas nas funcionalidades essenciais e ocultar os detalhes da implementação, o código se torna mais flexível e fácil de entender. Para um jovem desenvolvedor, isso significa que você pode criar sistemas mais complexos sem se perder em cada pequeno detalhe. Imagine que você está desenvolvendo um software de gerenciamento de veículos, onde diferentes tipos de veículos, como carros e caminhões, têm comportamentos distintos. Em vez de reescrever as mesmas funções em cada classe, você poderia usar a abstração para definir um comportamento comum na classe abstrata "Veiculo", deixando as particularidades para as subclasses. Dessa forma, você pode trabalhar de maneira mais eficiente e estruturada, com código mais claro e fácil de manter.

5.2.1 Classes Abstratas

Uma classe *abstrata* é uma classe que não pode ser instanciada diretamente. Ela serve como um modelo, definindo atributos e métodos comuns a um grupo de *subclasses*, mas deixando alguns métodos sem implementação (métodos abstratos), para que cada *subclasse* os defina de forma específica.

- **Exemplo:**

Código_5.1: Classe abstrata AnimalAbstracao em Java

```
1 // Classe abstrata AnimalAbstracao
2 inheritors
3 abstract class AnimalAbstracao {
4     private String nome;
5
6     public AnimalAbstracao(String nome){
7         this.nome = nome;
8     }
9
10    public String getNome() {
11        return nome;
12    }
13
14    // Método abstrato (sem implementação)
15    2 implementations
16    public abstract void emitirSom();
17 }
```

Código_5.2: Classe concreta CachorroAbstracao em Java

```
1 // Subclasse (classe filha)
2 public class CachorroAbstracao extends AnimalAbstracao {
3     public CachorroAbstracao(String nome){
4         super(nome); /* Chama o construtor da superclasse
5                        para inicializar o atributo nome. */
6     }
7     // Sobrescrevendo o método emitirSom da superclasse.
8     @Override
9     public void emitirSom(){
10         System.out.println("Au, au!");
11     }
12 }
```

Código_5.3: Classe concreta GatoAbstracao em Java

```
1 // Subclasse (classe filha)
2 public class GatoAbstracao extends AnimalAbstracao {
3     public GatoAbstracao(String nome){
4         super(nome); /* Chama o construtor da superclasse
5                        para inicializar o atributo nome. */
6     }
7     // Sobrescrevendo o método emitirSom da superclasse.
8     @Override
9     public void emitirSom() { System.out.println("Miau!"); }
12 }
```

Código_5.4: Classe principal TestaAnimalAbstracao em Java

```

1  public class TestaAnimalAbstracao {
2      public static void main(String[] args) {
3          AnimalAbstracao cachorro = new CachorroAbstracao( nome: "Billy");
4          AnimalAbstracao gato = new GatoAbstracao( nome: "Bibi");
5
6          cachorro.emitirSom(); // Au, au!
7          gato.emitirSom(); // Miau.
8      }
9  }

```

Saída:

```

Au, au!
Miau!

```

Nesse exemplo, a classe *AnimalAbstracao* é uma abstração: representa a ideia geral de um animal, sem se preocupar com o tipo específico. Já as classes *CachorroAbstracao* e *GatoAbstracao* fornecem as implementações concretas de *emitirSom()*.

5.2.2 Interfaces

Interfaces são outro mecanismo para abstração. Elas definem um contrato (um conjunto de métodos) sem fornecer implementação. Qualquer classe que “implemente” uma interface se compromete a fornecer o comportamento definido por ela.

- **Exemplo:**

Código_5.5: Classe interface em Java

```

1  // Abstração - interfaces
2  2 implementations
3  interface FormaInterfaces {
4      2 implementations
5      double calcularArea();
6  }

```

Código_5.6: Classe CirculoInterfaces em Java

```
1  class CirculoInterfaces implements FormaInterfaces {  
2      private double raio;  
3  
4      public CirculoInterfaces(double raio) {  
5          this.raio = raio;  
6      }  
7  
8      @Override  
9      public double calcularArea(){  
10         return Math.PI * raio * raio;  
11     }  
12 }
```

Código_5.7: Classe RetanguloInterfaces em Java

```
1  class RetanguloInterfaces implements FormaInterfaces {  
2      private double largura;  
3      private double altura;  
4  
5      public RetanguloInterfaces(double largura, double altura){  
6          this.largura = largura;  
7          this.altura = altura;  
8      }  
9  
10     @Override  
11     public double calcularArea() { return largura * altura; }  
14 }
```

Código_5.8: Classe TestaFormaInterfaces em Java

```

1 public class TestaFormaInterfaces {
2     public static void main(String[] args) {
3         FormaInterfaces circulo = new CirculoInterfaces( raio: 2);
4         FormaInterfaces retangulo = new RetanguloInterfaces( largura: 3, altura: 4);
5
6         System.out.println("Área do círculo: " + circulo.calcularArea());
7         System.out.println("Área do retângulo: " + retangulo.calcularArea());
8     }
9 }

```

Saída:

Área do círculo: 12.566370614359172

Área do retângulo: 12.0

Ao usar a interface Forma, estabelecemos a abstração de uma forma geométrica. Não importa se é um círculo, um quadrado ou um triângulo, basta saber que toda forma pode calcular sua área. Os detalhes internos (como se calcula a área) ficam a cargo das classes concretas.

5.3 Representação da Abstração em UML

No diagrama de classes UML, a abstração é representada por classes abstratas e interfaces, que são fundamentais para mostrar como os detalhes de implementação podem ser ocultados, enquanto os comportamentos essenciais são expostos. Uma classe abstrata é representada por um retângulo, assim como as classes comuns, mas seu nome é precedido pela palavra "abstract" ou, em alguns casos, o nome da classe é escrito em itálico. Além disso, métodos abstratos, que não possuem implementação na classe, são indicados com o nome seguido de um "?" ou com o modificador "abstract" em sua assinatura. Essas classes servem como um modelo para outras classes, que herdarão seus comportamentos e poderão implementar ou redefinir os métodos conforme necessário.

As interfaces, outro recurso importante para a abstração em UML, são representadas por um retângulo com o nome da interface precedido pela palavra "interface". Diferente das classes abstratas, as interfaces não podem ter implementação de métodos, apenas assinaturas. As classes concretas que

implementam a interface são conectadas a ela por uma linha sólida e com um triângulo vazio na extremidade, representando que a classe segue os contratos da interface. Ao usar diagramas de classe UML para ilustrar a abstração, o desenvolvedor pode visualizar claramente como a lógica do sistema é estruturada, sem se preocupar com a implementação dos detalhes de cada componente. Isso facilita a compreensão do sistema como um todo, permitindo que se concentre nas interações entre as classes e interfaces, em vez de se perder em implementações específicas.

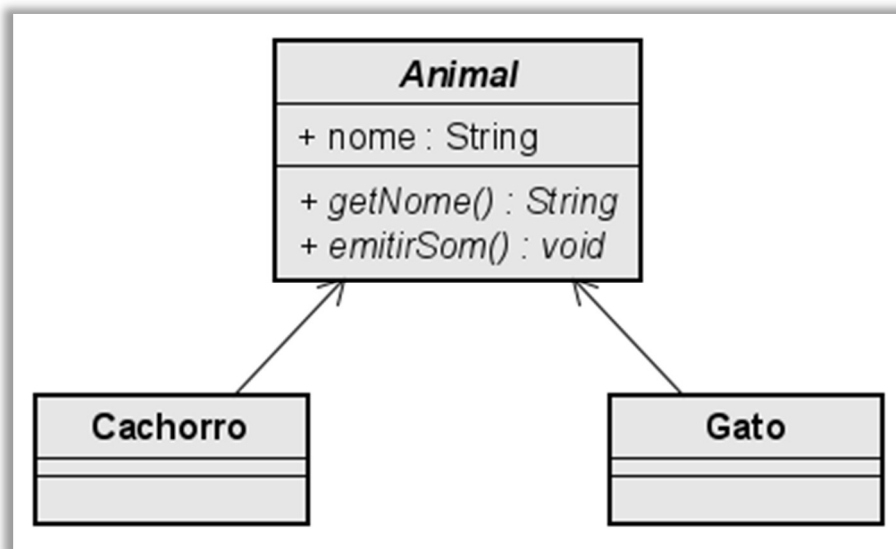
No diagrama de classes UML, a abstração se reflete de duas formas principais:

5.3.1 Classes Abstratas:

- O nome da *classe abstrata* é geralmente escrito em itálico.
- *Métodos abstratos* também podem ser apresentados em itálico ou sublinhados para enfatizar sua natureza.

Exemplo UML:

Imagem_5.1: Diagrama UML de Classe representando as classes Animal, Cachorro e Gato.



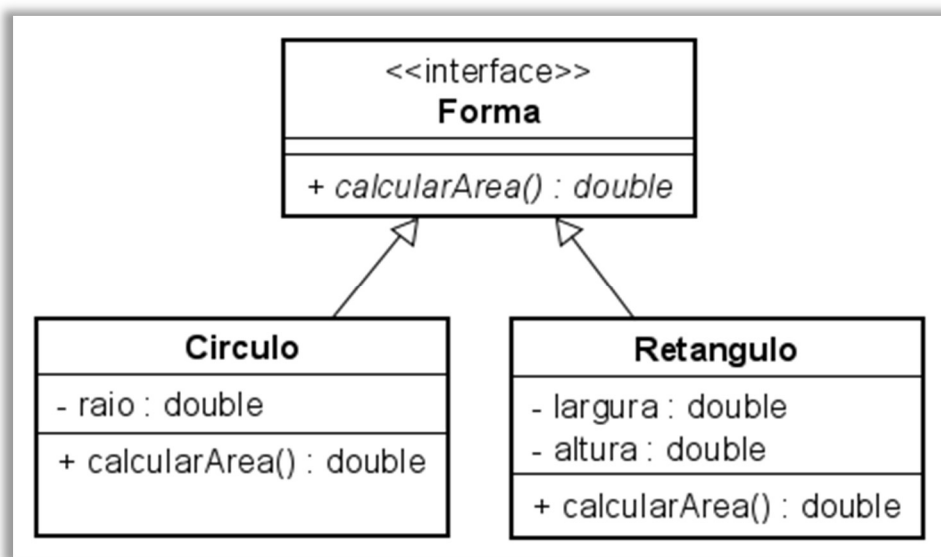
Aqui, a classe *Animal* é *abstrata*, enquanto *Cachorro* e *Gato* são classes *concretas* que *implementam* o método *emitirSom()*.

5.3.2 Interfaces:

- Representadas semelhante a classes, mas com a palavra «interface» no diagrama.
- Não possuem atributos ou métodos concretos, apenas a assinatura.

Exemplo UML:

Imagem_5.2: Diagrama UML de Classe representando as classes *Forma*, *Circulo* e *Retangulo*.



As setas com triangulação oca indicam que *Circulo* e *Retangulo* *implementam* a *interface* *Forma*.

Exemplos Práticos



- **Sistema de Biblioteca:** Uma classe *abstrata* `Publicacao` pode representar livros e revistas, permitindo que *subclasses* implementem detalhes específicos como periodicidade ou número de páginas.
- **Sistema de Gerenciamento de Veículos:** A classe *abstrata* `Veiculo` pode ser estendida para criar classes como `Carro` e `Moto`, cada uma implementando a funcionalidade mover de forma distinta.

Resumo do Capítulo



- **Abstração:** Um mecanismo para simplificar a compreensão de um sistema, destacando apenas o que é essencial.
- **Classes Abstratas:** Fornecem uma base genérica para subclasses, permitindo a implementação de comportamentos específicos.
- **Interfaces:** Definem contratos que as classes concretas devem cumprir, permitindo flexibilidade e intercambialidade de implementações.
- **UML:** Representa visualmente a abstração, destacando classes abstratas e interfaces, ajudando a compreender e comunicar a arquitetura do sistema.

Exercícios



1. **Explique Abstração:** Em suas próprias palavras, defina abstração e descreva como ela ajuda a resolver problemas de complexidade no desenvolvimento de software.
2. **Classes Abstratas x Interfaces:** Cite uma situação prática em que você usaria uma classe abstrata e outra em que escolheria uma interface. Explique suas razões.
3. **Criação de uma Hierarquia:** Desenhe um diagrama de classes (à mão ou usando software) representando uma hierarquia de veículos: Veículo (abstrato), Carro e Moto. Inclua métodos abstratos no Veículo (por exemplo, mover()) e implemente-os nas subclasses.
4. **Exemplo em Código:** Escreva uma interface Calculadora com o método calcular(int a, int b):int. Crie duas classes que implementem essa interface: Soma (que retorna a+b) e Multiplicacao (que retorna a*b). Teste-as com alguns exemplos e descreva o resultado.
5. **Metáfora para Abstração:** Crie uma nova metáfora (diferente das já apresentadas) para explicar o conceito de abstração a um amigo que está começando a estudar programação. Por que essa metáfora facilita a compreensão?

Conclusão



Com este capítulo, você aprendeu a importância da abstração, a representou em código e visualizou sua aplicação em UML. Seguindo esse raciocínio, o próximo passo é aprofundar-se nos outros pilares da Orientação a Objetos, garantindo assim uma compreensão sólida e completa desse poderoso paradigma de desenvolvimento.

Ao dominar a abstração, você dá um passo fundamental rumo a um design limpo, claro e fácil de manter. Esse é apenas um dos pilares da Orientação a Objetos: nos próximos capítulos, veremos outros conceitos igualmente importantes, como o Encapsulamento, a Herança e o Polimorfismo.